Fake News/Misinformation Detection (NLP)
(Week 2 Checkpoint Report)

Adrian John C. Alfonso
Computer Science Undergraduate
Mapaumanhin St. Manibaug Paralaya
Porac, Pampanga
+63 9476175343
adrianjohnalfonso47@gmail.com

Laurenzo S. Centeno
Computer Science Undergraduate
Jesus Street, Pulung Bulu
Angeles City, Pampanga
+63 926 022 8287
centenoenzo054@gmail.com

Seane Karl S. Garcia
Computer Science Undergraduate
San Roque St., Sta Cruz
Porac, Pampanga
+63 9396039256
grcseane@gmail.com

John Michael Y. Supan
Computer Science Undergraduate
Anita Road, Essel Park Subdivision City of San Fernando
+63 9610042777
supanjohn13@gmail.com

1. Introduction and Objectives

For the second week of development, the primary objective was to transition from the planning phase to active codebase construction. The team focused on securing a reproducible data pipeline, conducting exploratory data analysis (EDA), establishing traditional machine learning baselines, and scaffolding the advanced deep learning and reinforcement learning (RL) architectures. This report details the successful completion of these foundational milestones.

2. Data Pipeline and Exploration Analysis

To ensure pipeline reliability and avoid external library dependencies, the data acquisition script (`get_data.py`) was engineered to bypass Hugging Face. It directly downloads the official LIAR dataset from the UC Santa Barbara repository, extracts the raw TSV files, and cleans the directory locally. This approach guarantees reproducibility without committing large datasets to version control.

```
import os
import urllib.request
import zipfile

def download_and_save_data():
    print("Downloading the official raw LIAR dataset directly from UCSB...")

    # Define paths
```

```
    output_dir = os.path.dirname(os.path.abspath(__file__))
    zip_path = os.path.join(output_dir, "liar_dataset.zip")

    # Official dataset URL
    url = "https://www.cs.ucsb.edu/~william/data/liar_dataset.zip"

    # Download the zip file
    urllib.request.urlretrieve(url, zip_path)
    print("Download complete. Extracting raw TSV files...")

    # Extract the files
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(output_dir)

    # Rename 'valid.tsv' to 'val.tsv' to maintain pipeline consistency
    valid_tsv = os.path.join(output_dir, "valid.tsv")
    val_tsv = os.path.join(output_dir, "val.tsv")
    if os.path.exists(valid_tsv):
        os.rename(valid_tsv, val_tsv)

    # Clean up the zip file and remove the README
    os.remove(zip_path)
    readme_path = os.path.join(output_dir, "README")
    if os.path.exists(readme_path):
        os.remove(readme_path)

    print(f"Raw TSV data successfully extracted and cleaned up in {output_dir}")

if __name__ == "__main__":
    download_and_save_data()
```

This script acts as the automated data preparation engine for the machine learning models. It reads the raw TSV dataset, assigns proper column names, and filters out missing rows. It then standardizes the text by forcing lowercase and stripping punctuation, maps the original six-point truth scale into a clean binary format (0 for Fake, 1 for Real), and exports the finalized dataset as CSV files.

```
import pandas as pd
import re
```

```
import os

# PART 1: TOP OF FILE
DATA_DIR = "data/"
RAW_FILES = {
    "train": "train.tsv",
    "val": "val.tsv",
    "test": "test.tsv"
}

def clean_text(text):
    if not isinstance(text, str):
        return ""
    text = text.lower()
    text = re.sub(r"[^a-zA-Z0-9\s.,!?]", "", text)
    text = " ".join(text.split())
    return text

def map_binary_labels(label):
    fake_labels = ['pants-on-fire', 'false', 'barely-true']
    real_labels = ['half-true', 'mostly-true', 'true']
    if label in fake_labels: return 0
    if label in real_labels: return 1
    return None

def process_dataset():
    # PART 2: INSIDE THE FUNCTION
    columns = [
        'id', 'label', 'statement', 'subject', 'speaker', 'job',
        'state', 'party', 'barely_true_counts', 'false_counts',
        'half_true_counts', 'mostly_true_counts', 'pants_on_fire_counts', 'context'
    ]

    for split, filename in RAW_FILES.items():
        file_path = os.path.join(DATA_DIR, filename)

        if not os.path.exists(file_path):
            print(f"Warning: {file_path} not found.")
```

```
            continue

        # Updated loading logic for .tsv files
        df = pd.read_csv(file_path, sep='\t', names=columns, quoting=3)

        # Cleaning steps
        df = df.dropna(subset=['statement'])
        df['label'] = df['label'].apply(map_binary_labels)
        df['statement'] = df['statement'].apply(clean_text)
        df = df.dropna(subset=['label'])
        df['label'] = df['label'].astype(int)

        # Save as CSV for the Model Engineer
        output_path = os.path.join(DATA_DIR, f"cleaned_{split}.csv")
        df[['statement', 'label']].to_csv(output_path, index=False)
        print(f"Successfully saved {output_path}")

if __name__ == "__main__":
    process_dataset()
```

The EDA notebook (01_eda.ipynb) was developed to process these raw TSV files. The original six-point truth scale was collapsed into a strict binary classification target: 0 for Fake and 1 for Real. Missing text values were dropped, and sequence lengths were calculated to inform upcoming transformer tokenization limits.

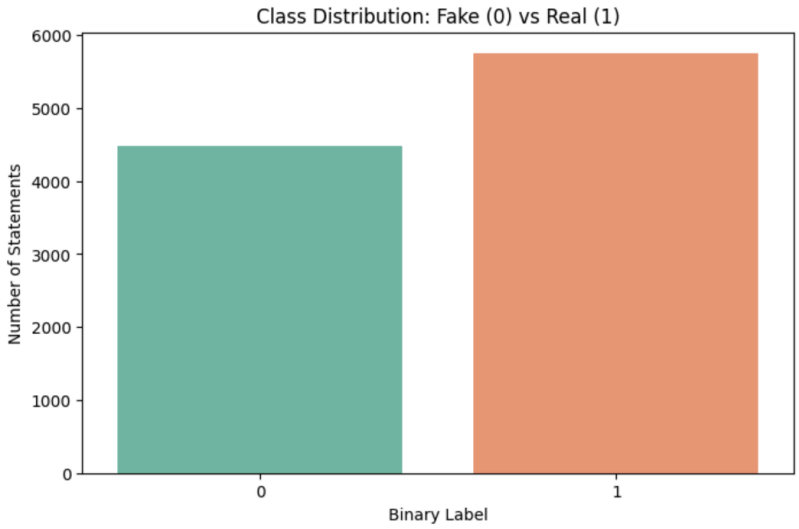

Figure 1: The class distribution of the binary mapped LIAR dataset, showing the balance between Fake (0) and Real (1) statements.

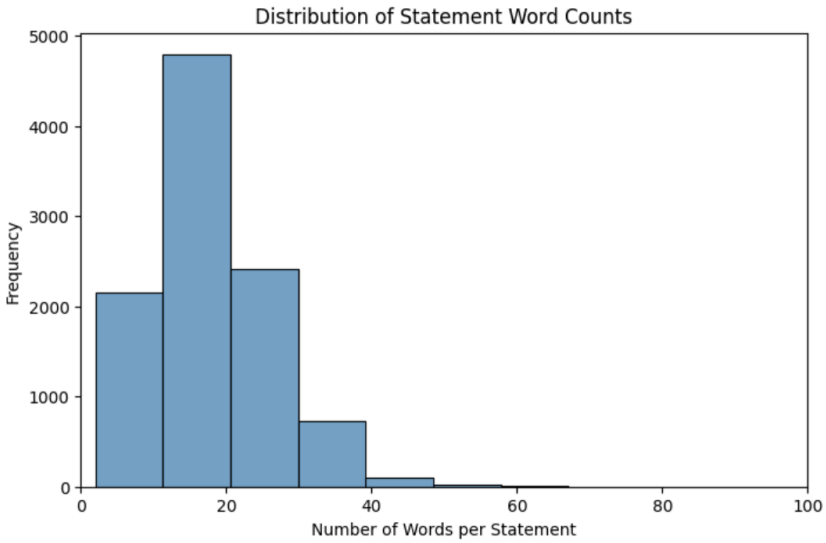

Figure 2: Distribution of statement word counts, utilized to set the `max_length` parameter for the DistilBERT tokenizer.

3. Machine Learning and DL Prototyping

Before training resource-intensive neural networks, a traditional machine learning baseline was established using Scikit-Learn. The cleaned text was vectorized using TF-IDF and fitted to a Logistic Regression model to capture our initial Accuracy and Macro F1 scores.

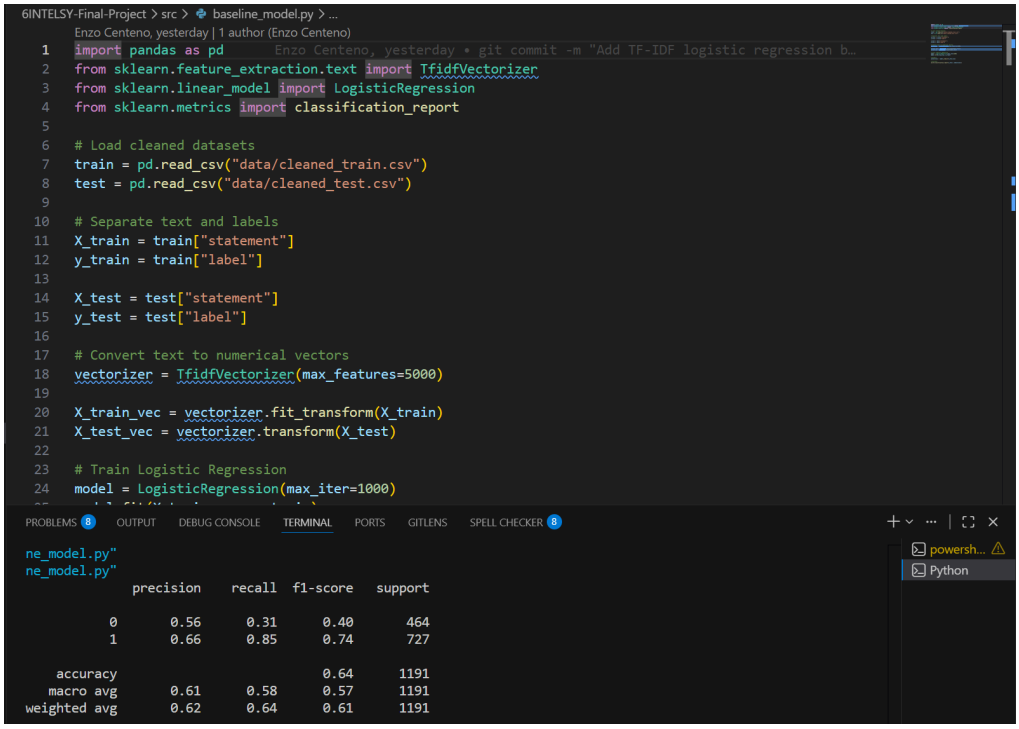

Figure 3: Initial classification report for the TF-IDF and Logistic Regression baseline.

Simultaneously, the core PyTorch architectures were prototyped in the `src/models/` directory. A 1D-Convolutional Neural Network (CNN) was built using `nn.Embedding` and `nn.Conv1d` layers to capture local linguistic features. A preliminary training loop was executed to verify that the model compiles and effectively learns from the batches. Furthermore, the Hugging Face `transformers` library was used to scaffold the DistilBERT class, preparing the pipeline for full fine-tuning in the upcoming phase.

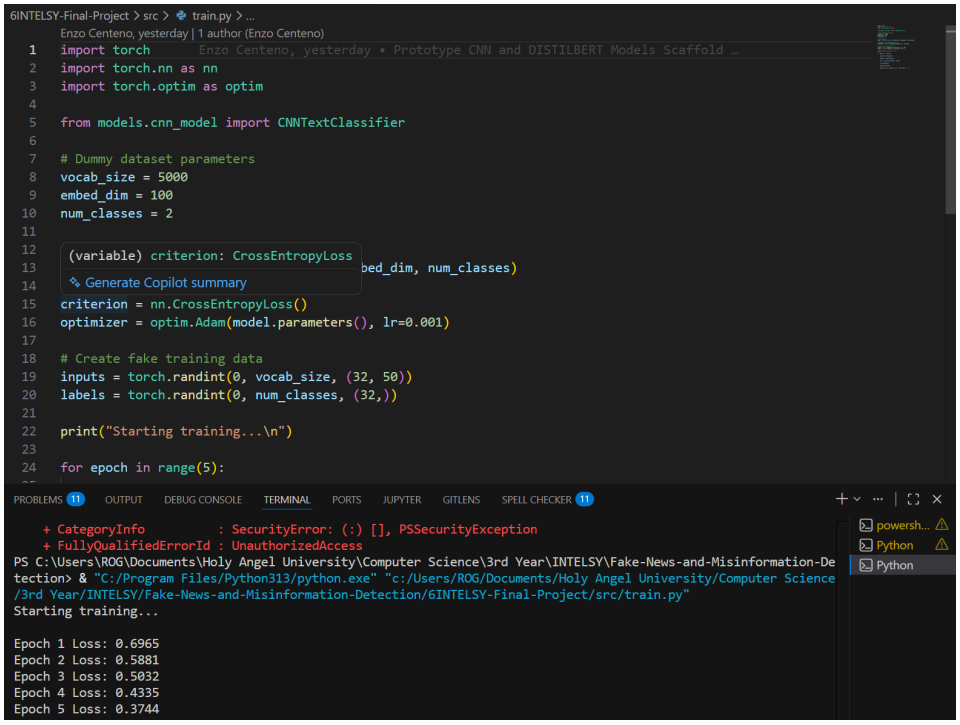

Figure 4: Early training loss curve for the PyTorch 1D-CNN prototype, demonstrating successful gradient descent.

4. Reinforcement Learning Environment Stub

A custom environment was built using the `gymnasium` library to house the Reinforcement Learning agent. The environment defines the "action space" as the adjustment of the classification probability threshold, and the "reward" as the resulting Macro F1 score evaluated on the validation set. A preliminary script executing random threshold actions over 50 episodes was run to validate the environment mechanics and generate an early learning curve.

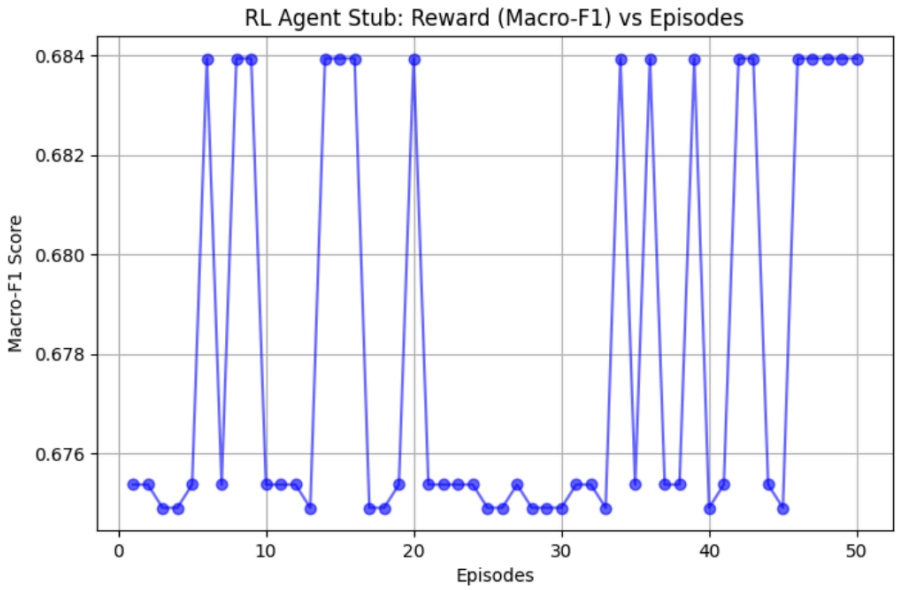

Figure 5: Early, unoptimized learning curve of the RL agent interacting with the threshold environment.

5. Ethical Considerations & Risk Mitigation

The team has initiated an active Risk Register to track ethical concerns. To mitigate algorithmic bias, the model prioritizes the Macro F1 metric to ensure equitable performance across majority and minority classes. To address the risk of over-censorship (false positives), the RL agent dynamically optimizes the decision threshold rather than relying on a rigid cutoff. Finally, strictly utilizing the public LIAR dataset ensures compliance with data privacy standards by avoiding the scraping of personally identifiable information (PII).